

Regiones de Convergencia en Sistemas No Lineales

Ruiz Luciana Gimena

13 de noviembre de 2024

Resumen

La resolución de problemas de aplicación que implican ecuaciones no lineales a menudo requiere el uso de métodos numéricos. Entre estos, el método de Newton destaca por su versatilidad y rapidez, lo que lo convierte en una herramienta valiosa para aproximar soluciones de sistemas no lineales. La aplicación del método de Newton a ecuaciones con variables complejas es particularmente interesante en el estudio de fractales, como en el problema de Cayley, donde se generan figuras fractales a partir de la convergencia, divergencia y eficiencia del método. En este trabajo se explora el problema de Cayley mediante la generalización del método de Newton en el plano $\mathbb{R}^2$ y presenta algunos fractales obtenidos a partir de las iteraciones del método en el ámbito de los números complejos.

1. Introducción

Este trabajo se centra en implementar el método de Newton en Python, aplicándolo a sistemas no lineales en dos dimensiones. A través de esta implementación, se busca no solo resolver ecuaciones complejas, sino también visualizar las regiones de convergencia en un plano, lo que proporciona una comprensión más intuitiva del comportamiento del método.

Al graficar las regiones de convergencia para múltiples puntos iniciales, se podrán observar patrones interesantes que revelan cómo el método de Newton interactúa con diferentes estructuras de ecuaciones. Esta visualización es crucial, ya que permite identificar las áreas desde las cuales el método converge rápidamente hacia soluciones, así como las zonas donde puede fallar o divergir.

Para lograr esto, dividiremos a dicha función en su parte real y su parte imaginaria, llevando el problema a $\mathbb{R}^2$, allí podremos utilizar el método de Newton de la forma que lo conocemos. Si bien el método puede ser aplicado directamente a funciones complejas esto escapa al enfoque de este trabajo.

Este trabajo esta basado en el siguiente [artículo](#) [1].

2. Trabajo realizado

Dada una función diferenciable $f : \mathbb{R} \rightarrow \mathbb{R}$, uno de los objetivos comunes es encontrar una raíz, es decir, un valor donde la función se iguala a cero.

El método de Newton es una técnica eficaz para lograr esto, comenzando con una buena aproximación inicial.

La fórmula de iteración se expresa como:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Sea $F \in \mathbb{R}^2$,

$$F(x, y) = \begin{bmatrix} h(x, y) \\ g(x, y) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

queremos encontrar (x^*, y^*) tal que $F(x^*, y^*) = \begin{bmatrix} h(x^*, y^*) \\ g(x^*, y^*) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

Para abordar este problema, utilizamos el polinomio de Taylor de h centrado en (x_1, y_1)

$$h(x, y) = h(x_1, y_1) + \nabla h(x_1, y_1) \begin{pmatrix} x - x_1 \\ y - y_1 \end{pmatrix} + \frac{\nabla^2 h(\varepsilon)}{2!} \begin{pmatrix} x - x_1 \\ y - y_1 \end{pmatrix}^2$$

Ahora supongamos (x_1, y_1) cercanos a (x^*, y^*) y $\nabla^2 h(x, y)$ acotado,

$$\begin{aligned} 0 &\approx h(x_1, y_1) + \nabla h(x_1, y_1) \begin{pmatrix} x^* - x_1 \\ y^* - y_1 \end{pmatrix} \\ 0 &\approx h(x_1, y_1) + (x^* - x_1) \frac{\partial h}{\partial x}(x_1, y_1) + (y^* - y_1) \frac{\partial h}{\partial y}(x_1, y_1) \end{aligned} \tag{1}$$

De forma análoga obtenemos para la función g :

$$0 \approx g(x_1, y_1) + (x^* - x_1) \frac{\partial g}{\partial x}(x_1, y_1) + (y^* - y_1) \frac{\partial g}{\partial y}(x_1, y_1)$$

Juntando ambas,

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} h(x_1, y_1) \\ g(x_1, y_1) \end{bmatrix} + \begin{bmatrix} \frac{\partial h}{\partial x}(x_1, y_1) & \frac{\partial h}{\partial y}(x_1, y_1) \\ \frac{\partial g}{\partial x}(x_1, y_1) & \frac{\partial g}{\partial y}(x_1, y_1) \end{bmatrix} \begin{bmatrix} x^* - x_1 \\ y^* - y_1 \end{bmatrix}$$

Método de Newton. Si $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ es diferenciable en x^k , para todo x en un entorno de x^k podremos aproximar $F(x)$ por $F(x^k) + F'(x^k)(x - x^k)$. De esta manera, dado x^k definiremos x^{k+1} como la solución de $F(x)$ por

$$F(x^k) + F'(x^k)(x - x^k) = 0.$$

Equivalentemente, $x^{k+1} = x^k + d^k$ donde d^k es la solución del sistema lineal

$$F(x^k) + F'(x^k)d = 0.$$

Con esto no solo aproximamos soluciones de sistemas no lineales, sino también las de funciones de variable complejas.

Ahora veamos como adaptar el método a una función compleja.

Podemos preguntarnos por las 3 raíces complejas del número 1, es decir, buscar las soluciones de la ecuación $z^3 - 1 = 0$, o lo que es igual, hallar las raíces del polinomio $z^3 - 1$, o de manera más general, las n raíces del polinomio $z^n - 1 = 0$

Trabajemos con la función $f(z) = z^3 - 1$.

Sea $z = x + iy$ obtenemos,

$$f(z) = f(x + iy) = x^3 - 3xy^2 - 1 + (3x^2y - y^2)i$$

Consideremos:

$$h(x, y) = x^3 - 3xy^2 - 1, \text{ la parte real de } f(z)$$

$$g(x, y) = 3x^2y - y^2, \text{ la parte imaginaria de } f(z)$$

¿Cómo es la cuenca de atracción de la raíz 1? es decir ¿qué puntos del plano tienen una órbita que converge a 1?

Para visualizar ésta y las otras dos cuencas de atracción tomamos la siguiente estrategia:

Implementacion en python.

Dado que para iterar es necesario calcular el Jacobiano, en Python se trabaja con expresiones simbólicas para poder diferenciar funciones. Para ello, se utilizó **Sympy**.

En esta parte del código armó $F(x^k)$ y $F'(x^k)$ que es la matriz jacobiana.

```
1 def F(x0):
2     x,y = x0
3     f1 = x**3 - 3*x*y**2 - 1
4     f2 = 3*x**2*y - y**3
5     return np.array([f1,f2])
6
7 #DEFINO MATRIZ JACOBIANA
8 def J(x0):
9     x = sym.Symbol('x')
10    y = sym.Symbol('y')
11    f1 = sym.sympify(x**3 - 3*x*y**2 - 1)
12    f2 = sym.sympify(3*x**2*y - y**3)
13
14    #calculo la derivadas parciales
15    df1x = sym.diff(f1, x)
16    df1y = sym.diff(f1, y)
17    df2x = sym.diff(f2, x)
18    df2y = sym.diff(f2, y)
19    # Evaluar el Jacobiano en el punto x0
20    J = np.array([
21        [df1x.subs({x: x0[0], y: x0[1]}), df1y.subs({x: x0[0], y: x0[1]})],
22        [df2x.subs({x: x0[0], y: x0[1]}), df2y.subs({x: x0[0], y: x0[1]})]
23    ], dtype= float)
24    return J
```

Aquí simplemente evalúe en cada iteración hasta llegar a un error determinado.

```
1     ...
2     b = F(x)
3     A = J(x)
4
5     #resolver sistema de ecuaciones
6     x_i = np.linalg.solve(A, -b)
7
8     #Actualizamos el punto
9     x = x + x_i
10    xn.append(x[0])
11    yn.append(x[1])
12    iteraciones = [k+1] + iteraciones
13
14    # Error
15    if norm(np.array(x)-[1,0],2) < tol:
16        break
17    if norm(np.array(x)-[-0.5,0.8660254],2) < tol:
18        break
19    elif norm(np.array(x)-[-0.5,-0.8660254],2) < tol:
20        break
```

Al punto le asocio el valor de la raíz correspondiente, donde le asigno un color a cada una de estas raíces, si converge a 1 se colorea de "tomato", a $-0.5 + 0.866025i$ de "navajowhite", a $0.5 - 0.866025i$ de "deeppink". (ver fig. 1)

Luego generalice esto creando una grilla de puntos iniciales en $\mathbb{R}^2$. Y usando el algoritmo para cada uno de ellos.

```
1     ...
2     for x0 in grilla:
3         solucion = newton(iteracion_jacobiano,x0)
4
5         real += solucion[0]
6         imaginaria += solucion[1]
7         color += solucion[2]
8         iteraciones += solucion [3]
```

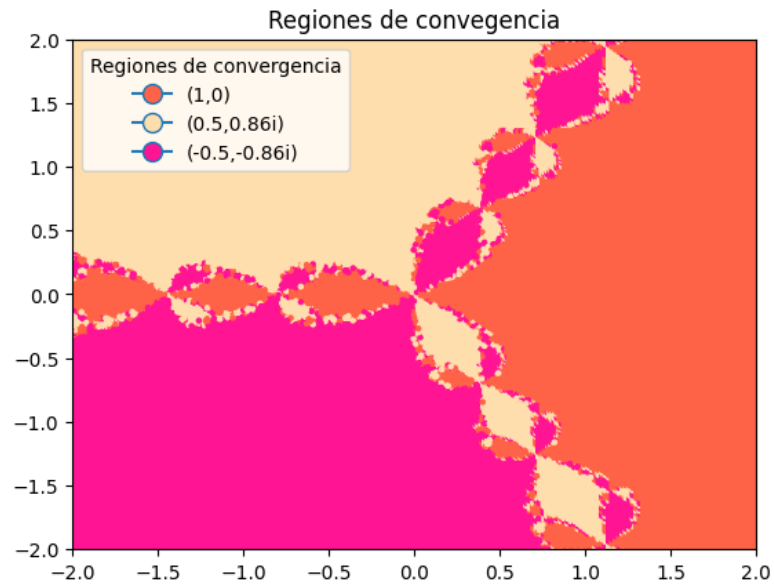


Figura 1: Cuencas de atracción para las tres raíces de $f(z) = z^3 - 1$.

Ahora a cada uno de los puntos, en lugar de asignarle el color según la raíz alcanzada por el método para ese punto, lo asignamos según la velocidad con la que cada uno de los puntos se acerca a su raíz, i.e., el número de iteraciones necesarias para converger o acercarse con la precisión deseada a la raíz. (ver fig. 2)

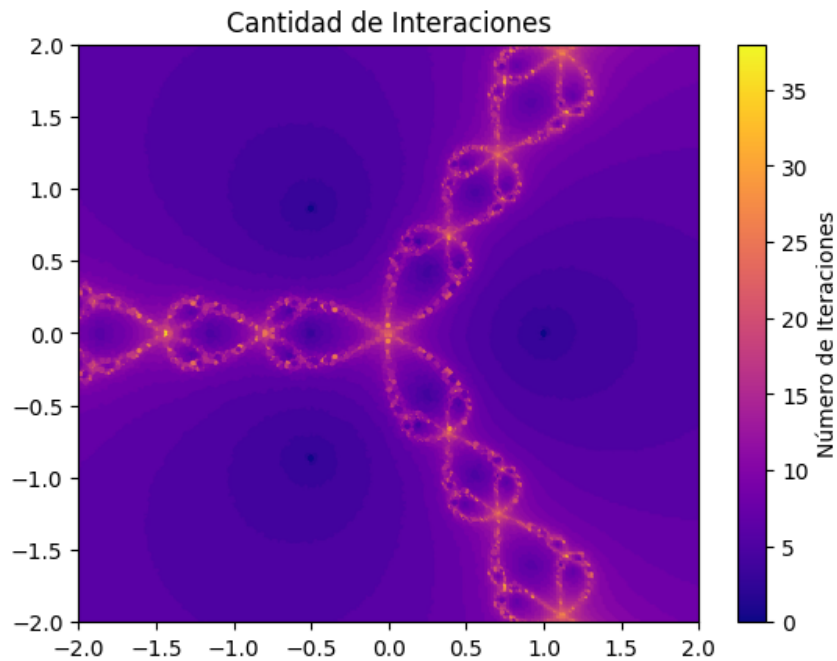


Figura 2: Velocidad de convergencia para las raíces de $f(z) = z^3 - 1$.

Las n raíces del polinomio $z^n - 1 = 0$.

Por la fórmula $z^n = r^n(\cos n\pi + i \sin n\pi)$ las n raíces del polinomio $z^n - 1 = 0$ están esparcidas en la circunferencia unidad.

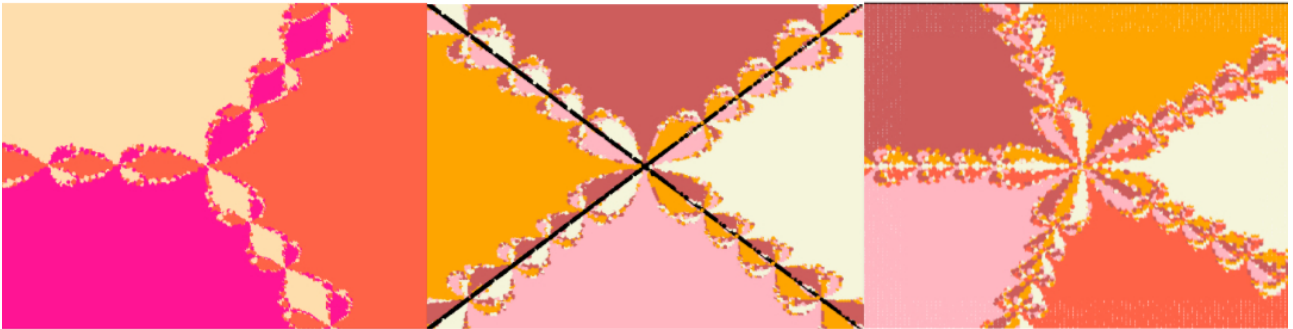


Figura 3: Cuencas de atracción para las n -raíces de 1 , $n = 3, 4, 5$.

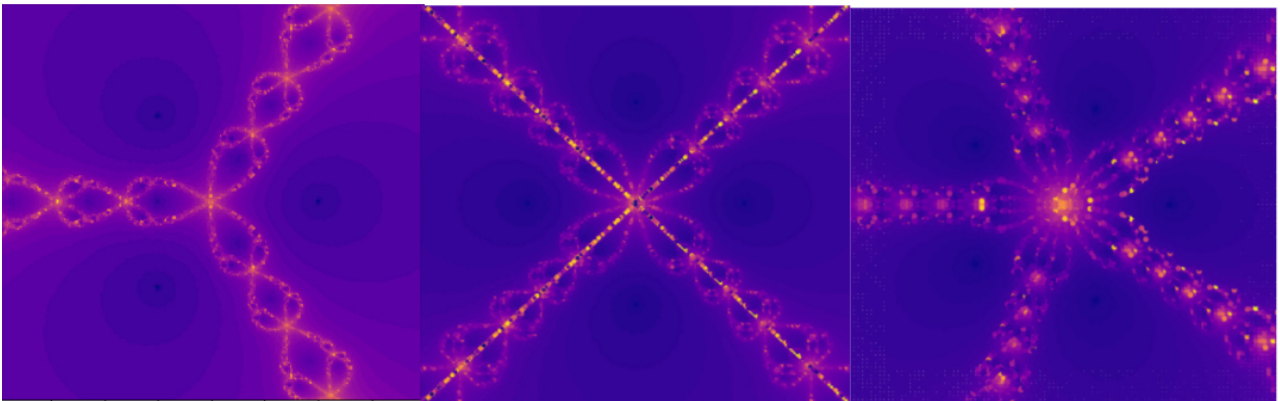


Figura 4: Velocidad de convergencia para las n -raíces de 1 , $n = 3, 4, 5$.

De manera similar, todo lo anterior lo podemos calcular para otros polinomios. Por ejemplo,

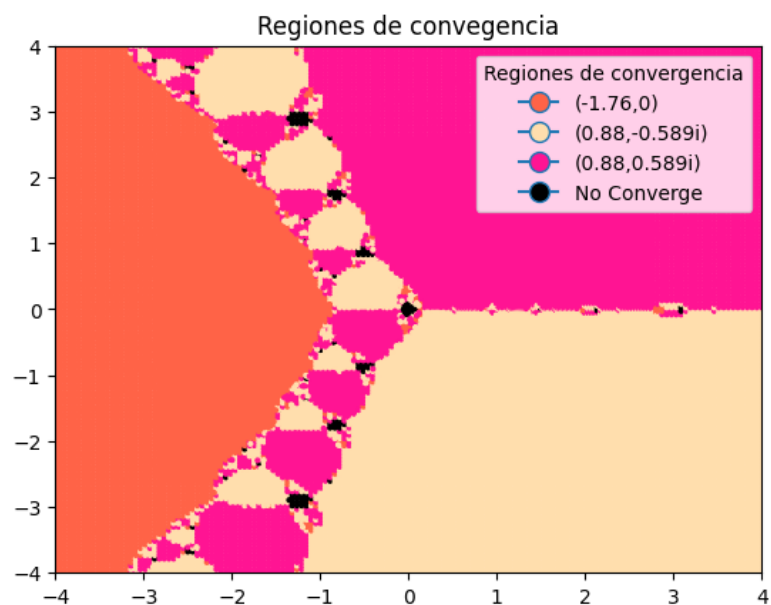


Figura 5: Cuencas de atracción para $z^3 - 2z + 2$

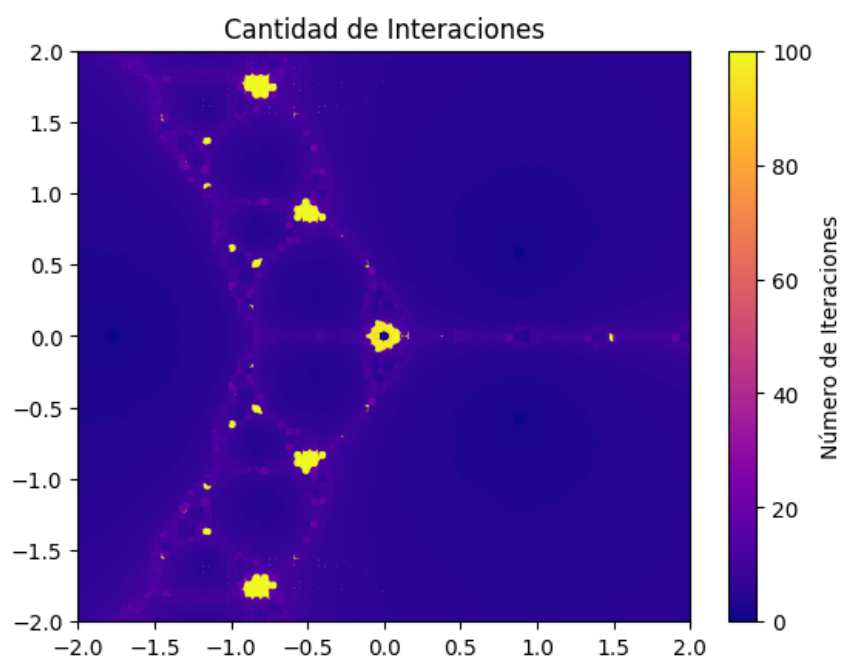


Figura 6: Velocidad de convergencia para las raíces de $z^3 - 2z + 2$

Pueden ver el código en este link: [Código](#)

A. Conclusión

En la figura 1 hemos pintado las cuencas de atracción, y se puede observar la frontera entre las cuencas de atracción de las tres raíces, la cual muestra la complejidad de esta región (ver fig. 2) —conjunto de Julia—.

También observamos que en los gráficos las figuras se repiten infinitamente y en todas las direcciones.

Cabe notar que en la mayoría de los casos se convergió muy rápido, por ejemplo, con un error del $1e-4$, el máximo número de iteraciones para $z^3 - 1$ fue: 38. O por ejemplo, con un error del $1e-6$, para $z^4 - 1$ fue: 70.

Referencias

- [1] Juneth Andrea Terán Tarapués y Catalina María Rúa Alvarez EL MÉTODO DE NEWTON PARA RAÍCES COMPLEJAS. FRACTALES EN EL PROBLEMA DE CAYLEY. Escuela de Ingeniería de Antioquia